

챕터 1. MSA란 무엇인가?

이번 챕터가 끝나면

- 모놀리식 아키텍처의 한계를 이해할 수 있습니다.
- 마이크로서비스 아키텍처가 어떤 방식으로 문제를 해결하는지 이해할 수 있습니다.
- 이 책에서 만들 쇼핑몰 주문 시스템의 전체 구조를 파악할 수 있습니다.
- MSA의 핵심 과제인 분산 트랜잭션과 그 해법(보상 트랜잭션·비동기 메시지)을 이해할 수 있습니다.
- 챕터 2~5의 학습 흐름을 파악할 수 있습니다.

준비하기. 챕터 2부터 시작될 실습을 위해 미리 준비

이 챕터는 개념만 다루므로 직접 코드를 작성하지 않습니다. 챕터 2부터 실습이 시작되니, 그전에 도구 설치와 레포 위치를 미리 확인해 두세요.

1. 실습 환경

도구	사용 챕터	설치 주소
Docker Desktop	챕터 2~	https://www.docker.com/products/docker-desktop/
Minikube	챕터 3~	https://minikube.sigs.k8s.io/
Hoppscotch (브라우저 확장)	챕터 2~	https://hoppscotch.io/

Docker Desktop을 설치하고 실행한 뒤 "Engine running" 상태인지 확인합니다.

2. 챕터별 소스 코드

이 책의 실습은 챕터마다 GitHub 레포가 하나씩 대응합니다. 챕터 2부터 해당 레포를 클론하여 진행합니다.

챕터	레포	주제
챕터 2	github.com/metacoding-12-msa/ex01	동기 REST + 보상 트랜잭션
챕터 3	github.com/metacoding-12-msa/ex02	DDD + 클린 아키텍처 + Kubernetes
챕터 4	github.com/metacoding-12-msa/ex03	Kafka + Orchestration Saga
챕터 5	github.com/metacoding-12-msa/ex04	WebSocket 실시간 알림

3. 진행 순서

챕터 2부터 챕터 5까지 한 시스템이 단계적으로 진화합니다. **챕터 2부터 순서대로 진행하세요.**

새벽 세 시. 휴대폰 알람이 울렸다. 가까운 데서 한 번, 멀리서 한 번. 침대 옆 협탁의 알람과 거실 노트북의 알람이 동시에 울려대고 있었다.

화면을 열었다. 모니터링 대시보드가 빨갛게 차 있었다. 에러 그래프가 가파른 절벽처럼 솟아 있었다. 주문 API가 죽었고, 그 옆에 로그인 API도 같이 죽어 있었다.

주문이랑 로그인이 무슨 상관이야?

서버 한 대를 재시작했다. 그래프가 잠시 떨어지더니 다시 솟았다. 또 한 대. 또. 마지막 한 대를 재시작하자 알람이 조용해졌다. 시계를 보니 새벽 네 시 반이었다.

월요일 아침, 팀장이 자리로 왔습니다.

팀장 : "왜 같이 죽었어요? 주문이 몰린 건데 로그인이 왜 같이 멈췄요? 방법 좀 찾아 봐요."

오픈이가 답을 못 했습니다. 서버를 다시 살린 건 자기지만, 왜 같이 죽었는지는 모릅니다. 사무실 끝에 앉은 선배 자리로 걸어갔습니다.

오픈이 : "선배님, 금요일에 주문이 몰렸는데 로그인까지 같이 죽었어요. 주문이랑 로그인이 무슨 상관인데 같이 멈추는 건지 모르겠어요."

선배 : "한 건물에 가게를 다 넣어두면 그래요. 한 층에서 불나면 전체가 대피해야 하잖아요."

1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

1.1.1 처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있습니다. 운영사 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리합니다. 이 구조는 단순하고 효율적입니다.



그림 1-1. 백화점 - 모든 매장이 한 건물에 모여 있는 구조

소프트웨어도 같은 이야기입니다. 처음에는 하나의 서버에 모든 기능을 넣는 **모놀리식** 구조가 단순하고 빠릅니다.

1.1.2 성장하면서 균열이 생긴다

백화점이 잘 돼서 방문객이 열 배로 늘었습니다. 이제 문제가 보이기 시작합니다. 한 매장에서 불이 나도 전 층이 영업을 멈춥니다. 한 매장에 손님이 몰려도 그 매장만 따로 늘릴 수 없으니 건물 전체를 새로 지어야 합니다. 전기 배선 한 번 교체하려면 공사 기간 내내 전 층이 닫힙니다.

소프트웨어 세계에서도 똑같은 일이 벌어집니다. 회원 10만 명, 하루 주문 1만 건이 되었을 때, 모놀리식 구조의 균열이 드러납니다.

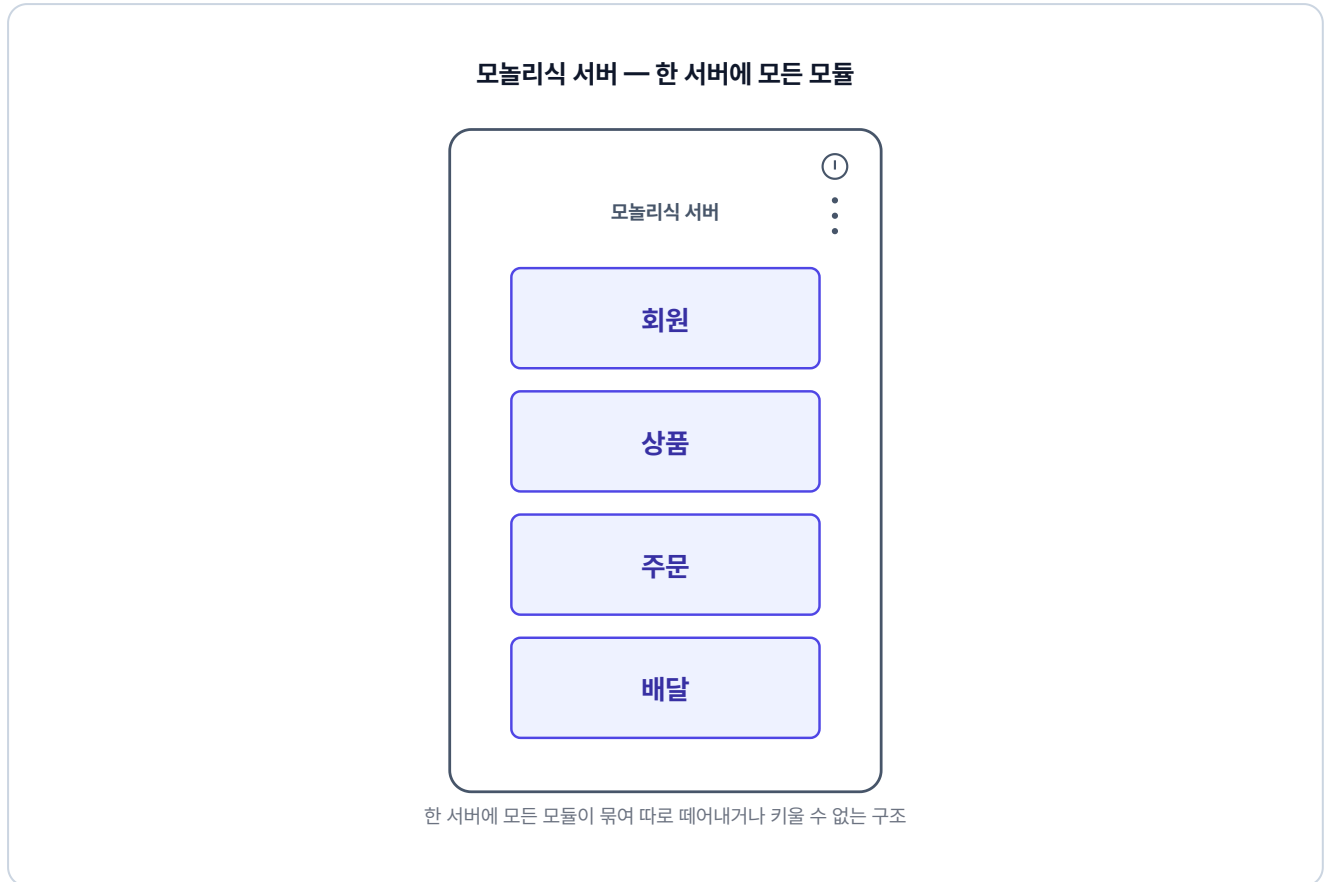


그림 1-2. 모놀리식 쇼핑몰 - 모든 기능이 하나의 서버에

배포가 두렵습니다. 주문 기능 하나를 수정해도 배포 시 회원·상품·배달까지 전부 재시작해야 합니다.

장애가 전파됩니다. 배달 기능 버그로 서버가 느려지면 배달과 무관한 회원 로그인도 함께 느려집니다.

확장이 어렵습니다. 블랙프라이데이에 주문 기능만 서버를 늘리고 싶어도 모놀리식에서는 전체 서버를 복제해야 합니다.

팀이 커지면 충돌이 잦아집니다. 10명이 하나의 코드베이스를 동시에 수정하면 하루에도 수십 번 충돌이 발생합니다.

이래서 서버가 전부 같이 죽었구나. 한 건물에 다 넣어뒀으니까.

오픈이 : "그럼 어떻게 해야 하나요?"

선배 : "백화점 대신 개별 상점을 열면 돼요. 각자 건물을 가지면 한 곳에서 불이 나도 나머지는 멀쩡하잖아요."

1.2 마이크로서비스 - 역할을 나눈다

1.2.1 백화점 vs 개별 상점

개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다.



그림 1-3. 개별 상점 - 각 매장이 독립된 건물로 운영되는 구조

백화점 구조와 비교하면 차이가 분명합니다.

	백화점	개별 상점
한 매장 화재 발생	스프링클러·대피령으로 전 층 영업 중단	해당 매장만 소방 처리, 나머지 정상 영업
전자제품 수요 폭증	건물 전체를 확장해야 함	전자제품점만 확장
건물 전기 배선 교체	공사 기간 동안 전 층 영업 중단	해당 매장만 임시 폐쇄, 나머지 정상 영업

마이크로서비스 아키텍처가 바로 이 방식입니다. 하나의 큰 서버 대신, 기능별로 작은 서비스들을 분리합니다.



그림 1-4. 마이크로서비스 쇼핑몰 - 기능별로 서비스를 분리

이제 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 주문 서비스에 버그가 생겨도 회원 서비스는 영향을 받지 않습니다.

1.2.2 MSA vs 모놀리식

특성	모놀리식	마이크로서비스
배포	전체 재배포	해당 서비스만 배포
장애 격리	전체 영향	해당 서비스만 영향
확장	전체 복제	필요한 서비스만 확장
팀 분리	하나의 코드베이스	서비스별 독립 개발

오픈이 : "우리 쇼핑몰로 치면 어떻게 나누면 되나요?"

선배 : "회원, 상품, 주문, 배달. 이 네 개부터 떼어내 봐요."

1.3 시스템과 핵심 과제

문제를 이해했으니, 이제 직접 만들어볼 시스템을 설계해 보겠습니다.

1.3.1 만들 시스템

이 책의 쇼핑몰 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

서비스	포트	역할
회원 서비스	8083	로그인, JWT 발급, 사용자 조회
상품 서비스	8082	상품 목록, 재고 조회 및 증감
주문 서비스	8081	주문 생성·조회·취소 (핵심)
배달 서비스	8084	배달 생성·조회·취소

주문 서비스가 중심에서 다른 서비스를 엮습니다. 사용자가 주문하면 상품 서비스의 재고를 차감하고, 배달 서비스에 배달을 만듭니다.

1.3.2 분산 트랜잭션 — 이 책의 핵심 과제

서비스를 분리하면 곧장 새로운 문제가 생깁니다. 모놀리식에서는 `@Transactional` 한 줄로 주문·재고·배달을 자동 롤백할 수 있었습니다. 그런데 MSA에서는 각 서비스가 독립된 데이터베이스를 가집니다. 한쪽에서 일어난 일을 다른 쪽이 모르므로, 자동 롤백이 불가능합니다. 이렇게 여러 DB에 걸친 작업을 하나로 묶어야 하는 상황을 **분산 트랜잭션**이라고 합니다.



그림 1-5. 서비스별 독립 데이터베이스

분산 트랜잭션(Distributed Transaction)이란? 여러 독립된 데이터베이스에 걸친 작업을 하나의 논리적 단위로 처리해야 하는 상황입니다. MSA에서는 서비스마다 DB가 분리되어 있으므로 단일 트랜잭션이 불가능하고, 별도의 전략이 필요합니다.

재고는 줄였는데 배달이 실패하면... 자동으로 안 돌아간다고?

오픈이 : "선배님, 이거 되돌리는 방법이 없나요?"

선배 : "자동으로 안 되니까 직접 짜는 거죠. 재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고. 한 단계씩 거꾸로 되돌리는 거예요. 보상 트랜잭션이라고 해요."

보상 트랜잭션(Compensating Transaction)이란? 중간에 실패가 나면 이미 끝낸 작업을 역순으로 돌려 원래 상태로 되돌리는 방법입니다.

1.3.3 분산 트랜잭션을 푸는 두 가지 방식

이 책에서는 분산 트랜잭션을 두 가지 방식으로 풀어 갑니다.

방식 1. 직접 호출과 보상 (챕터 2, 챕터 3)

각 서비스가 다음 서비스를 **직접** 부릅니다. 주문 서비스가 상품 서비스에 "재고 줄여줘"를 요청해 처리하다가 뒤에서 실패하면, 다시 직접 "재고 돌려줘"를 요청해 되돌립니다. 호출자가 응답까지 멈춰 있는 동기 호출 방식이라, 한 호출이 1초 걸리면 사용자도 그만큼 기다립니다.



그림 1-6. 동기 REST - 서비스 간 직접 호출과 보상

단순한 구현이지만, 서비스 수가 늘어날수록 보상 코드가 깊어집니다.

방식 2. 메시지로 비동기 전환 (챕터 4, 챕터 5)

서비스끼리 직접 부르지 않고 메시지로 주고받습니다. 보내는 쪽은 결과를 기다리지 않으므로(비동기), 한 서비스가 느려도 다른 서비스가 멈추지 않습니다. 전체 흐름을 조율하는 별도 서비스는 뒤에서 자세히 다룹니다.



그림 1-7. 비동기 메시지 - 서비스를 분리하는 구조

지휘자가 전체 흐름을 알고 있어 실패 시 자동으로 롤백 명령을 내립니다. 두 방식을 모두 직접 구현해 보면, 각각의 장단점이 자연스럽게 체감됩니다.

1.4 이 책의 학습 흐름

이제 전체 그림이 보입니다. 이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 챕터는 이전 챕터의 한계를 느끼는 것에서 시작합니다.

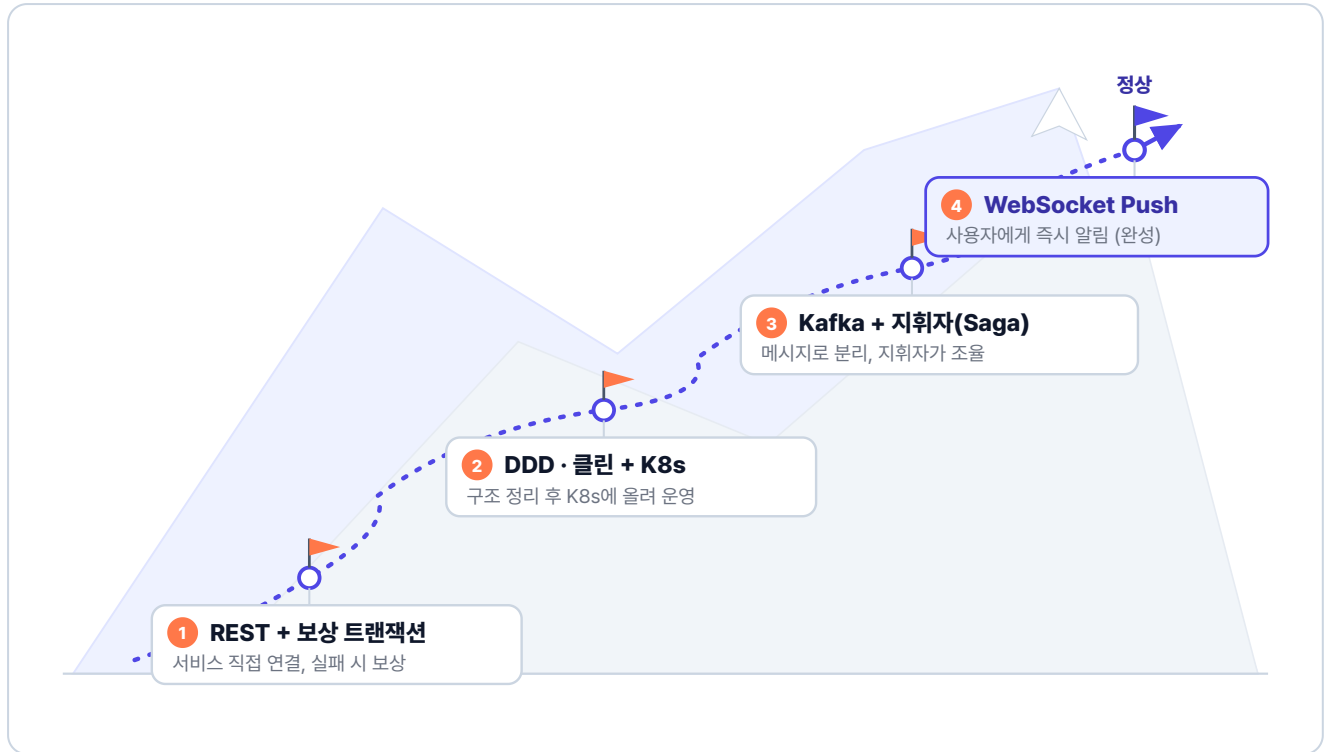


그림 1-8. 이 책의 학습 흐름

가장 단순한 방법, REST로 네 서비스를 직접 잇는 것에서 출발합니다. 뼈대가 서고 나면 호출이 뒤엉켜 손대기 어려워지므로, 도메인을 중심으로 구조를 정리해 쿠버네티스에 올립니다. 구조가 깔끔해져도 동기 호출은 서로를 기다리다 함께 멈추기에, 메시지로 서비스를 분리해 비동기로 바꿉니다. 끝으로 처리가 끝난 순간, 사용자에게 실시간으로 알립니다.

선배 : "개념은 이 정도면 됐어요. 이제 직접 만들어 봐요. 제일 단순한 방법부터."

손가락이 움직이지 않았던 게 엇그제인데, 이제 뭘 만들어야 할지는 보인다.

이것만은 기억하자

- **모놀리식**은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생깁니다.
- **마이크로서비스**는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 합니다.
- MSA의 핵심 과제는 **분산 트랜잭션**입니다. 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없습니다.
- 분산 트랜잭션을 두 가지 방식으로 풀니다. 챕터 2~3에서는 **보상 트랜잭션**으로 서비스끼리 직접 호출·보상하고, 챕터 4~5에서는 메시지로 비동기 통신해 서비스를 분리합니다.
- 이 책은 하나의 쇼핑몰 주문 시스템이 챕터 2부터 챕터 5까지 단계적으로 진화하는 이야기입니다.

이제 직접 코드를 작성할 시간입니다. 챕터 2에서는 4개의 서비스를 REST로 연결하고, 보상 트랜잭션을 구현해 봅니다. 처음에는 단순하게 시작합니다. 그 단순함이 나중에 어떤 문제를 만드는지, 챕터 2에서 직접 느껴 봅니다.